

Programovací jazyky a překladače

Přednáška 9, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

Dnešní témata

- Administrativa
 - Všechny zápočtové a zkouškové odkazy v Moodle jsou nebo budou včas dostupné
- Sémantická analýza
 - Syntaxí řízený překlad, atributové gramatiky a překladová schémata
 - Sémantická analýza v generátorech *yacc/Bison* (dokončení z PP8)
- Úvod do generování mezikódu ---> PREZENTACE
 - Graf (abstraktní syntaktický strom, AST, univerzální, výhodný zejména z hlediska optimalizace, s výhodou se váže na grafově orientované atributové gramatiky)
 - Tříadresový kód (registrově orientovaný)
 - Postfix (zásobníkově orientovaný)

Syntaxí řízený překlad

Sémantické (významové, výstupní) akce **synchronní se syntaktickou kontrolou**

- Zajišťují jednopřechodovost překladu
- Eliminují výrazové nedostatky bezkontextových gramatik
- Přiřazují hodnoty a další atributy pojmenovaným terminálním i *neterminálním* symbolům
 - Rozsah platnosti (scope)
 - Typová kompatibilita a další
 - Informace se průběžně ukládají do tabulky symbolů

Metody generování akcí

- **Syntaxí řízený překlad**

- Nejjednodušší, praktické, akce jsou obvykle na konci pravidel

- $E \rightarrow E_1 + T$ `{printf("ADD");}`

- **Atributové gramatiky**

- Formálně orientované, zavádějí atributy u gramatických symbolů a obvykle využívají topologické řazení pro jejich zpracování

- $E \rightarrow E_1 + T$ `{E.val = E1.val + T.val;}`

- **Překladová schémata**

- Prakticky orientované, umísťují akce na libovolná realistická místa pravidel

- $E \rightarrow$ `{printf("PUSH ");}` $E_1 + T$ `{printf("ADD");}`

- **Kombinace** všech výše uvedených možností

Syntaxí řízený překlad

Miroslav Beneš
Dušan Kolář



Výstupy obou typů analýz

1 + 2 * 3

- Výstup syntaktické analýzy: (konkrétní) syntaktický strom (**parse tree**)

– Pouze pravidla

$S \rightarrow ABC$

Pravidla gramatiky (NT)

- Výstup sémantické analýzy: (**abstraktní**) syntaktický strom (AST)

– Pravidla a akce

$S \rightarrow ABC \{ \dots \}$ MĚNA PRÁVIDLA, POZTE AKCE

- Jednorázově (na konci pravidla, S-atributové gramatiky)

- Vícenásobně (uvnitř i na konci pravidla, L-atributové gramatiky)

$E \rightarrow E + T$

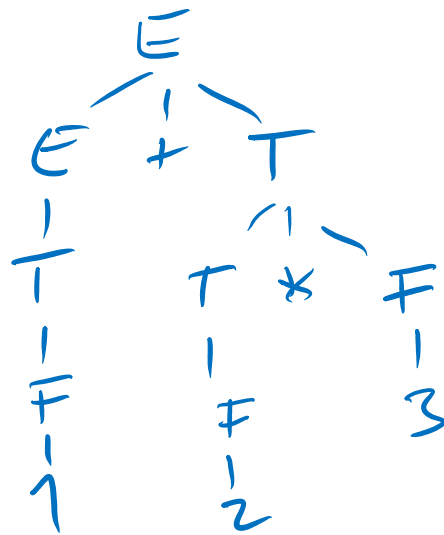
| T

$T \rightarrow T * F$

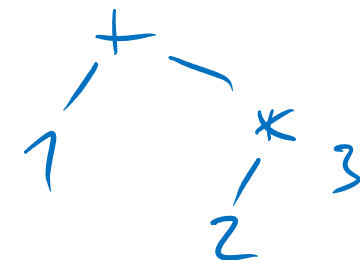
| F

$F \rightarrow \text{num}$

num



AST



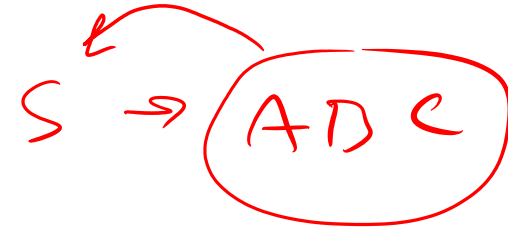
SEM. PRÁVIDLO

OKL
AD

DĚDÍCENÉ

Atributy

$$A \rightarrow X_1 X_2 \dots X_n$$



Syntetizované atributy:

- Atribut A je odvozen z atributů symbolů $X_1, X_2, \dots, X_n$

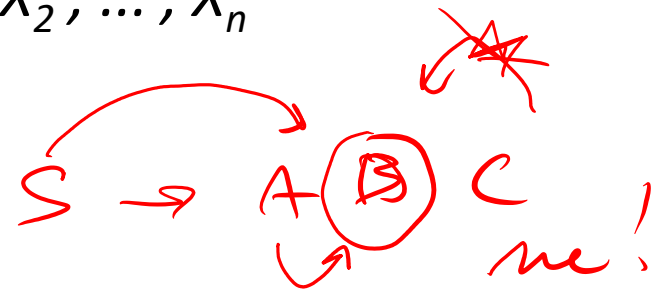
$$E_0 \rightarrow E_1 + T \quad \{ E_0.val \leftarrow E_1.val + T.val \}$$

Dědičné atributy

- Atribut $X_1, X_2, \dots, X_n$ je odvozen z atributů symbolů $A, X_1, X_2, \dots, X_n$

$$Decl \rightarrow Type L; \quad \{ L.type \leftarrow Type.type \}$$

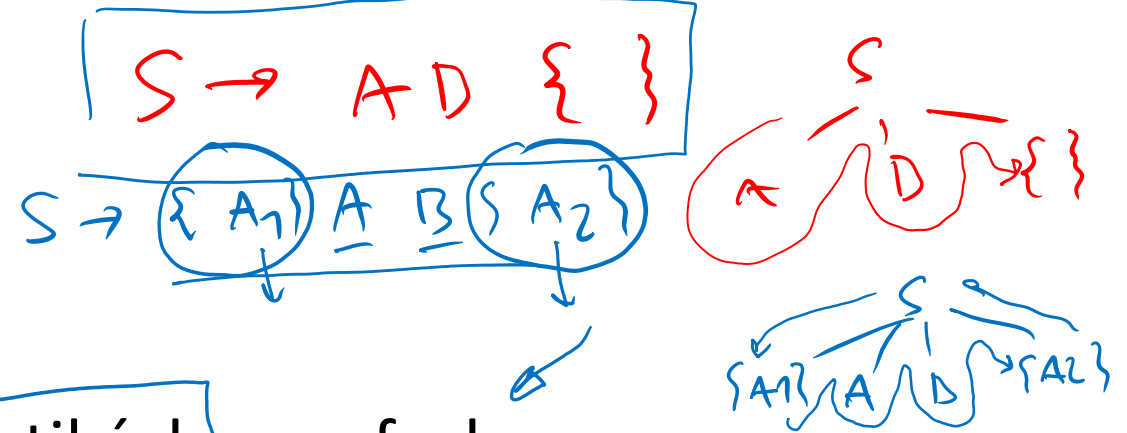
$$L_0 \rightarrow L_1, id \quad \{ L_1.type \leftarrow L_0.type \}$$



Nejčastější způsoby vyhodnocování atributů

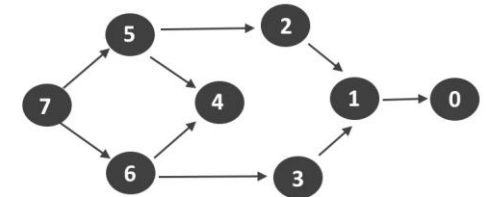
- Metody, využívající průchod stromem (Treewalk), akce jsou součástí stromu

1. Analýza pravidel v čase překladu
2. Stanovení pořadí pravidel
3. Zpracování pravidel v tomto pořadí



- Metody, založené na atributových gramatikách a grafech

1. Syntaktický strom (Parse Tree)
2. Graf závislostí (Dependency Graph) – přímý acyklický graf (DAG)
 - Viz např. utilita *make* nebo přepočítávání buněk v tabulkách
3. Topologické setřídění grafu závislostí
4. Vyhodnocení atributů v topologickém pořadí

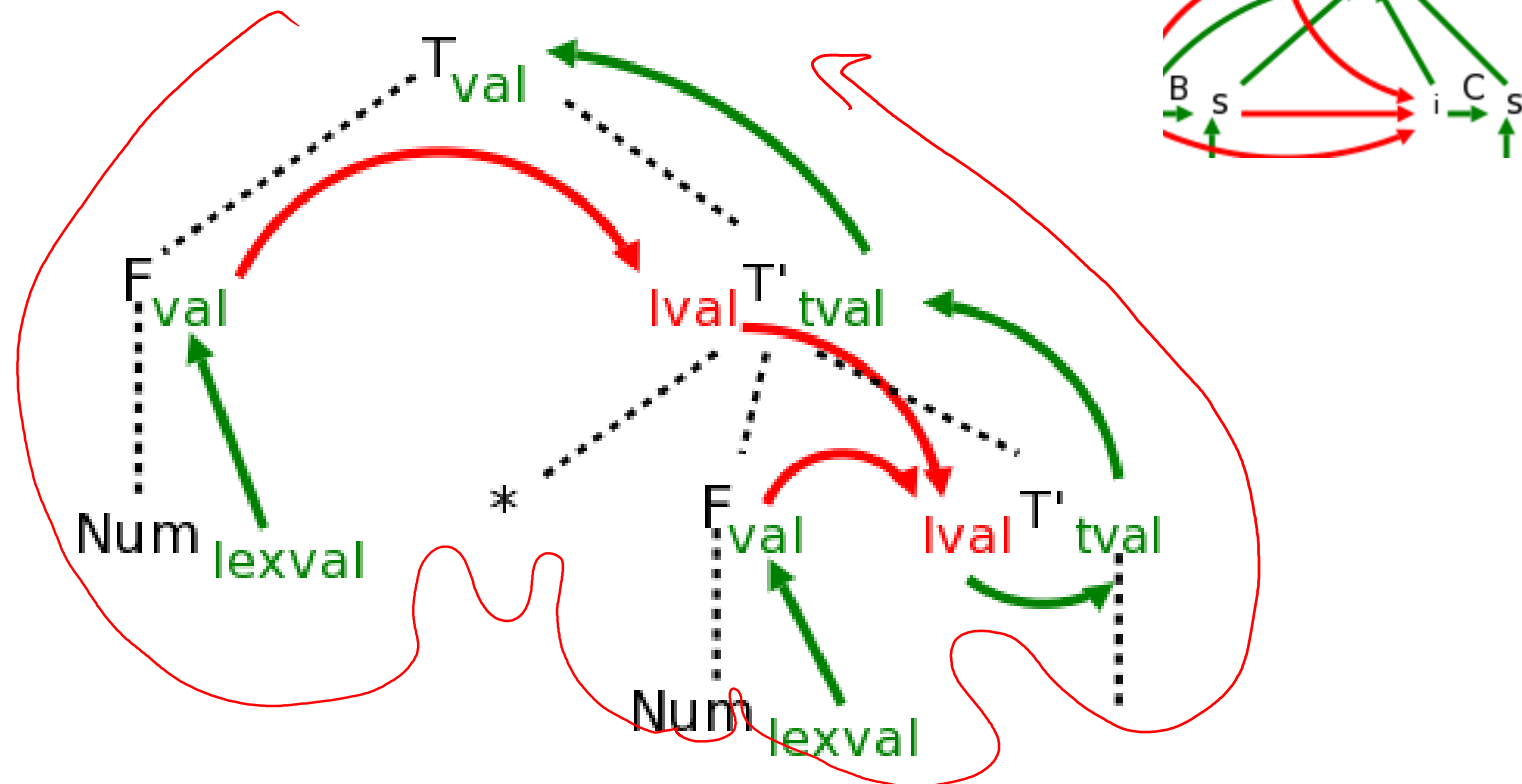


Topological Sort : 7 6 5 4 3 2 1 0

Pořadí vyhodnocování atributů

- Graf závislosti (dependency graph)
 - **Topologické třídění** (topological sort)
 - 1. Dědičné (červené šipky, zleva doprava, shora dolů)
 - 2. Syntetizované (zelené šipky, jen zdola nahoru)

Příklad:



Gramatika pro aritmetické výrazy, opakování a shrnutí

1. Víceznačná

$E \rightarrow E + E \mid E * E \mid (E) \mid n$

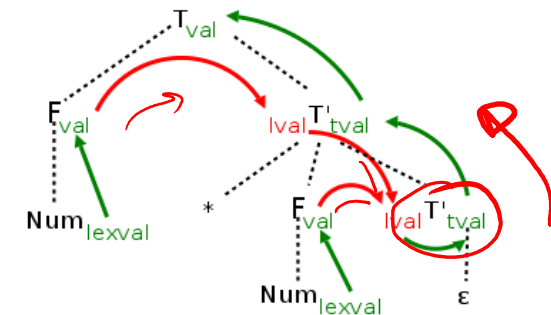
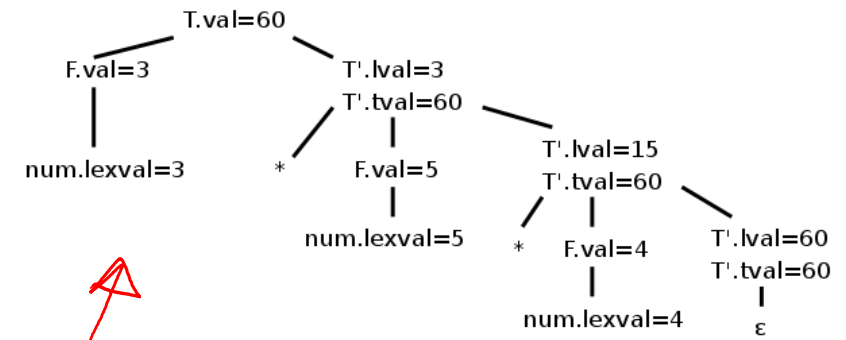
2. LR, S-atributová

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid n$

3. LL, L-atributová

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \varepsilon$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \varepsilon$
 $F \rightarrow (E) \mid n$

$T \rightarrow F T' \quad T'.lval = F.val$
 $\quad \quad \quad T.val = T'.tval$
 $T' \rightarrow * F T' \quad T'.lval = T'.lval * F.val$
 $\quad \quad \quad T'.tval = T'.tval$
 $T' \rightarrow \varepsilon \quad T'.tval = T'.lval$
 $F \rightarrow num \quad F.val = num.lexval$



Syntaxí řízený překlad zdola nahoru a shora dolů (postfix)

$E \rightarrow E + T$
 $\quad | T$
 $T \rightarrow T * F$
 $\quad | F$
 $F \rightarrow \text{num}$

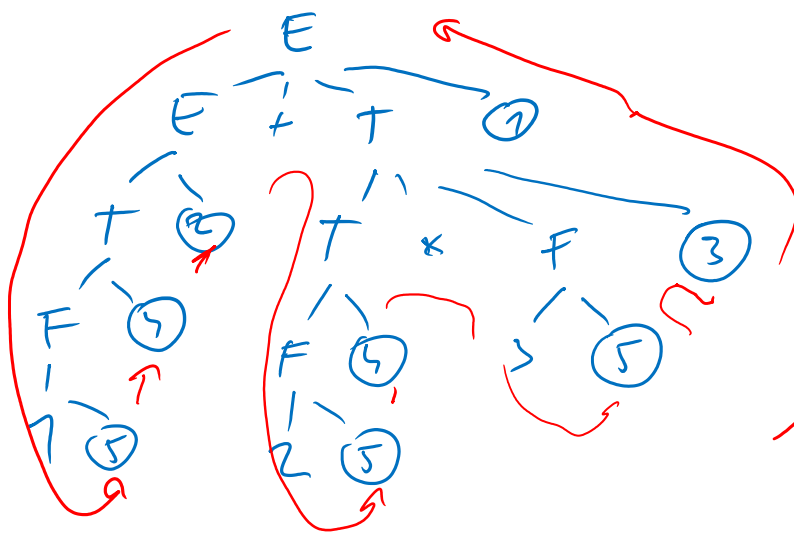
{ printf('+') } ①
{ } ②

{ printf('') }* ③
{ } ④

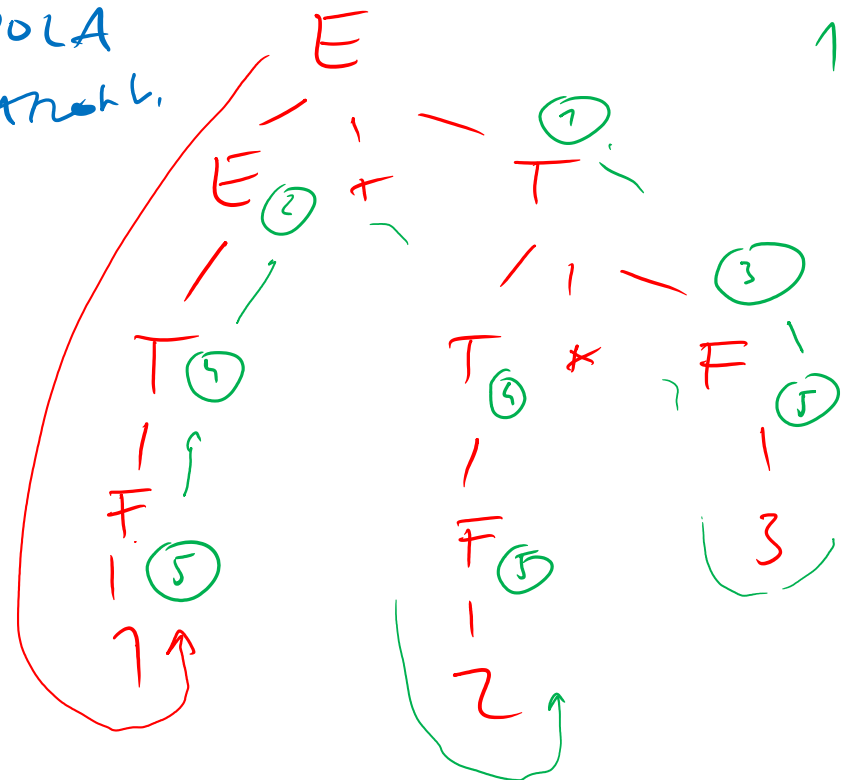
{ printf(num.val) } ⑤

shora dolů

1 + 2 * 3
 → 1 2 3 * +



zdola nahoru



1 2 3 * +

1 2 3 * +

S - ATN ID
 S - AD { _ }

Syntaxí řízený překlad, převod do postfixu

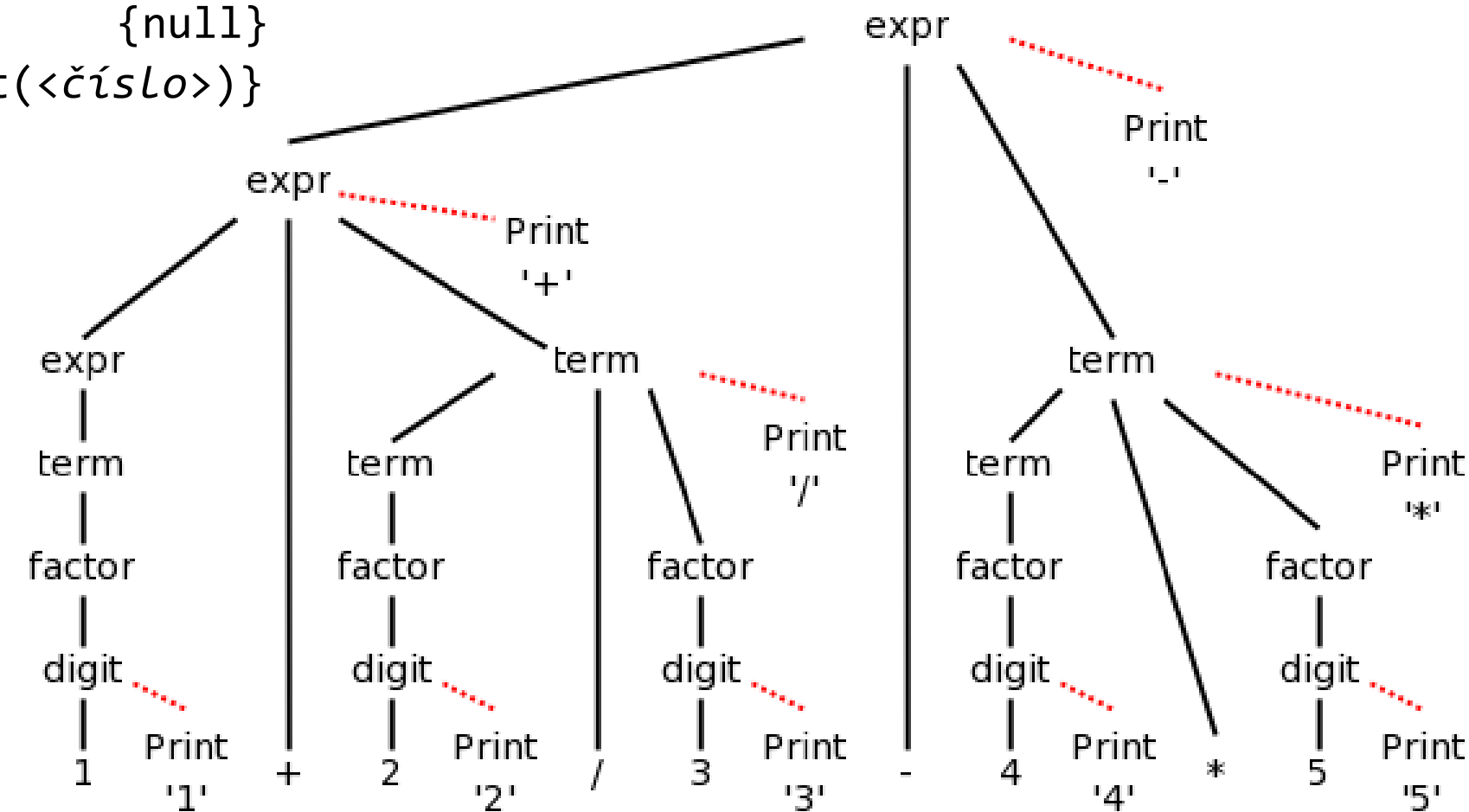
expr → expr + term {print('+')}

expr → expr - term {print('-')}

term → term / factor {print('/')}

term → factor {null}

digit → <číslo> {print(<číslo>)}



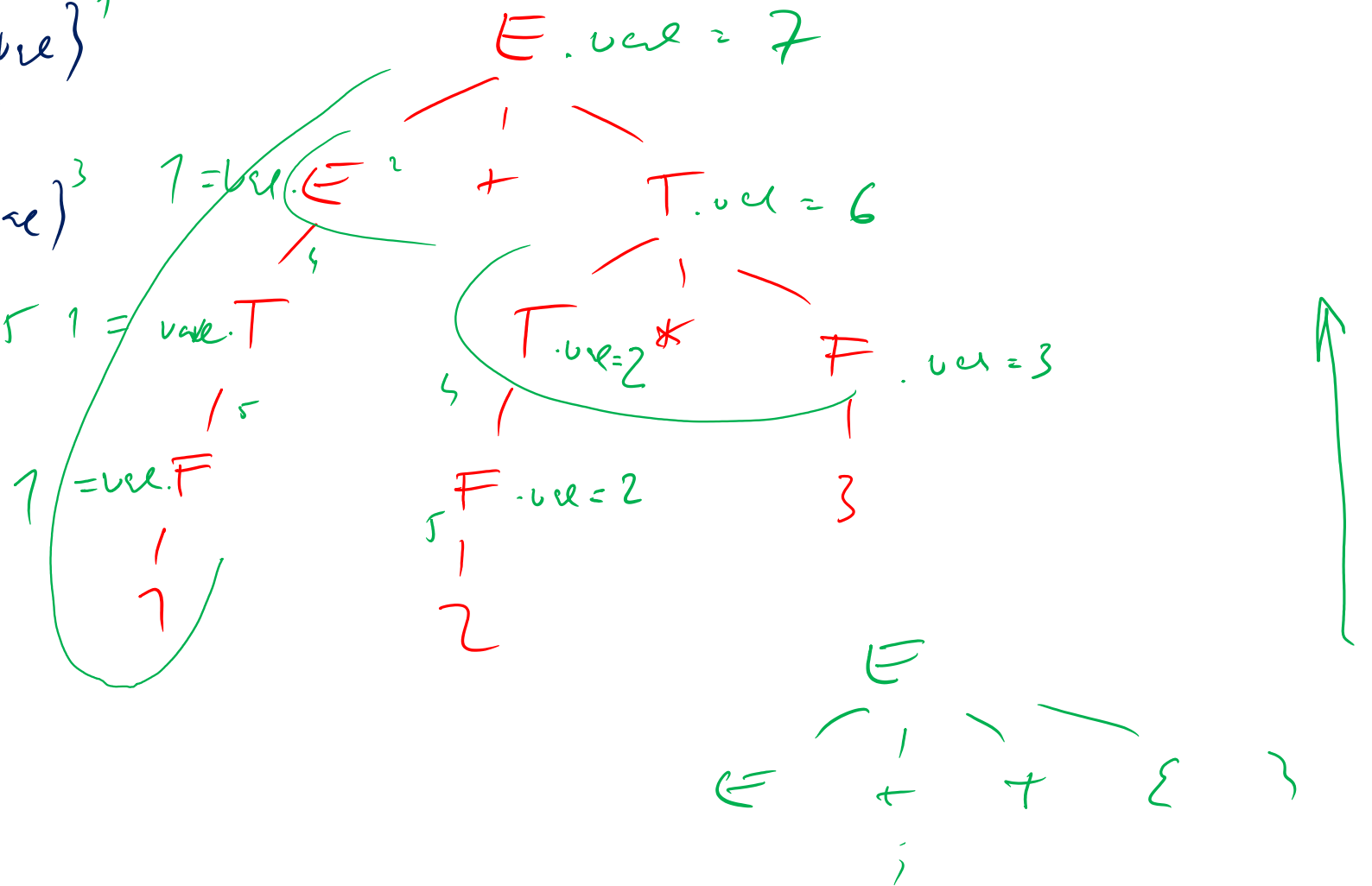
Syntaxí řízený překlad, hodnoty syntetizovaných atributů

$E \rightarrow E + T \quad \{E.val = E.val + T.val\}^1$
 $\quad \mid T \quad \{E.val = T.val\}^2$

$T \rightarrow T * F \quad \{T.val = T.val * F.val\}^3$
 $\quad \mid F \quad \{T.val = F.val\}^4$

$F \rightarrow num \quad \{F.val = num.val\}$

$9 + 5 * 6$
 $1 + 2 * 3$

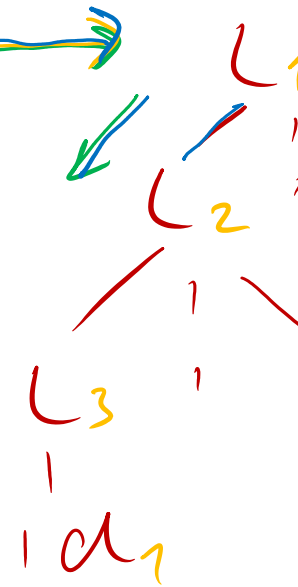
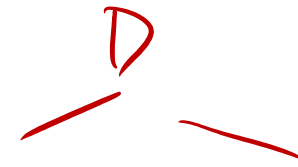
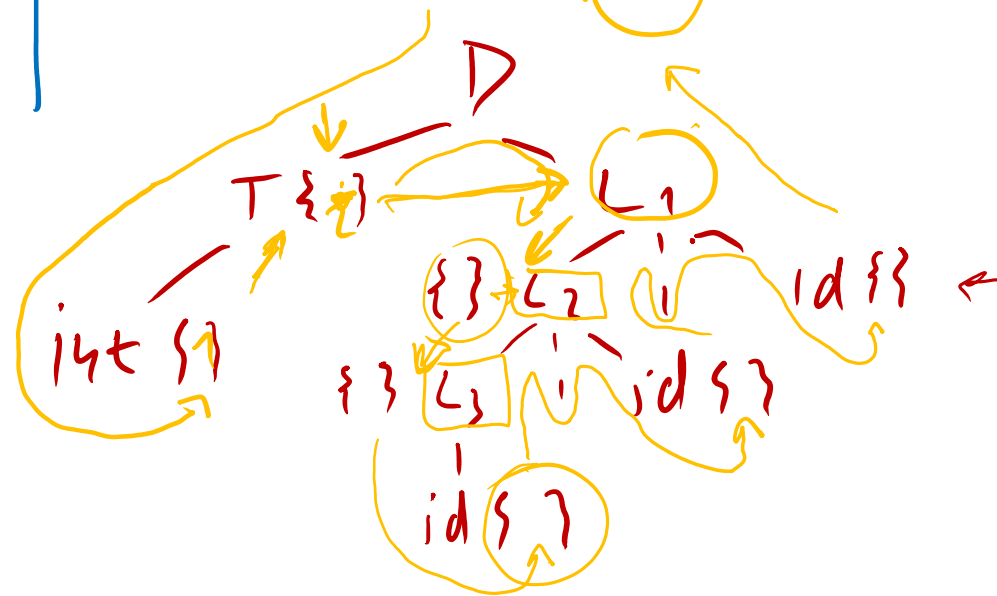


Dědičné atributy, příklad

$D \rightarrow TL$
 $T \rightarrow int$
 $T \rightarrow real$
 $L \rightarrow L_1 id$
 $L \rightarrow id$

$D \rightarrow T \{ L.is = T.type \} L$
 $T \rightarrow int \{ T.type = 'int' \}$
 $T \rightarrow real \{ T.type = 'real' \}$
 $L \rightarrow \{ L.is = L.is \} L_1, id \{ add = L.is \}$
 $L \rightarrow id \{ add = (id.add, L.is) \}$

int a, b, c



TOP. Soud

$id_1.add$
 $id_2.add$
 $id_3.add$

$T.type = int$

$L.is = T.type$

$addtype(id.add, L.is)$

$\rightarrow L2.is = L1.is$

$addtype(id.add, L2.is)$

$\rightarrow L3.is = L2.is$

$addtype(id.add, L3.is)$

$A \rightarrow BCD \{ \{ S$
 $A \rightarrow BCD$

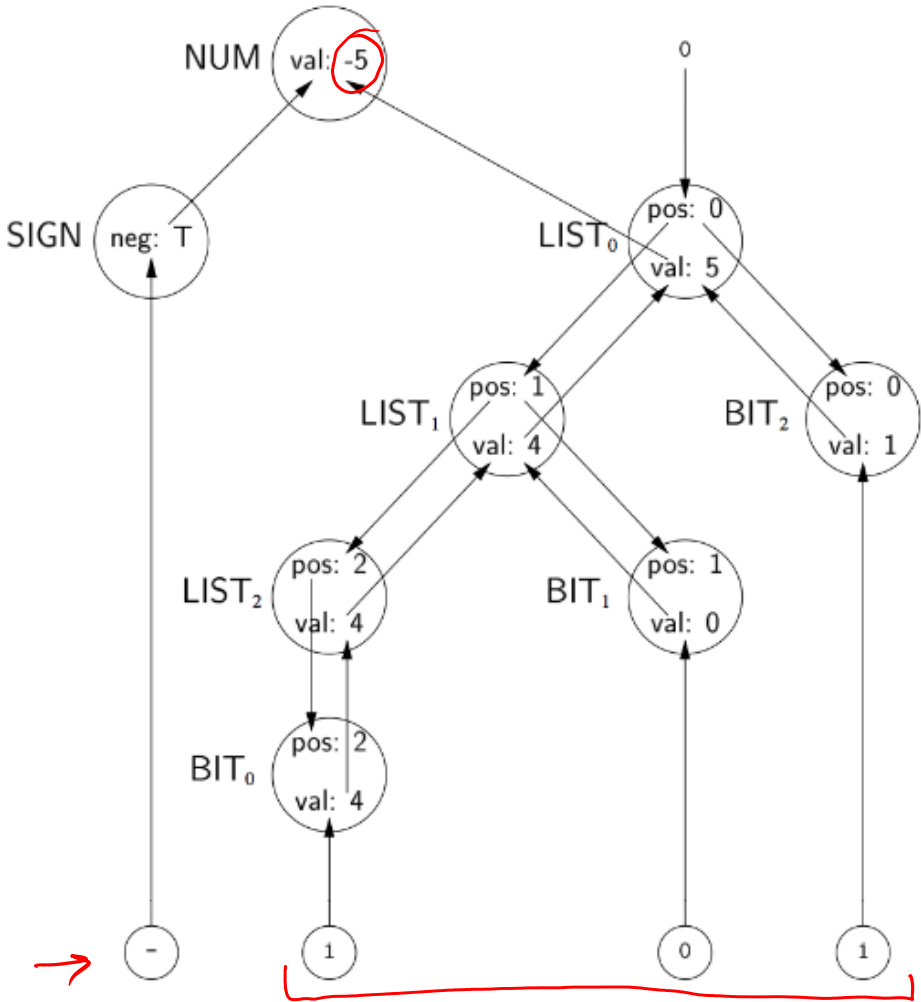
Atributová gramatika, příklad konverze sign. bin --> int

1. Atributová gramatika

Productions			Attribution Rules
<i>Number</i>	→	<i>Sign List</i>	$List.pos \leftarrow 0$ If <i>Sign.neg</i> then $Number.val \leftarrow -List.val$ else $Number.val \leftarrow List.val$
<i>Sign</i>	→	$\pm$	$Sign.neg \leftarrow false$
		$=$	$Sign.neg \leftarrow true$
$List_0$	→	$List_1 Bit$	$List_1.pos \leftarrow List_0.pos + 1$ $Bit.pos \leftarrow List_0.pos$ $List_0.val \leftarrow List_1.val + Bit.val$
		<i>Bit</i>	$Bit.pos \leftarrow List.pos$ $List.val \leftarrow Bit.val$
<i>Bit</i>	→	0	$Bit.val \leftarrow 0$
		1	$Bit.val \leftarrow 2^{Bit.pos}$

Symbol	Attributes
<i>Number</i>	val
<i>Sign</i>	neg
<i>List</i>	pos, val
<i>Bit</i>	pos, val

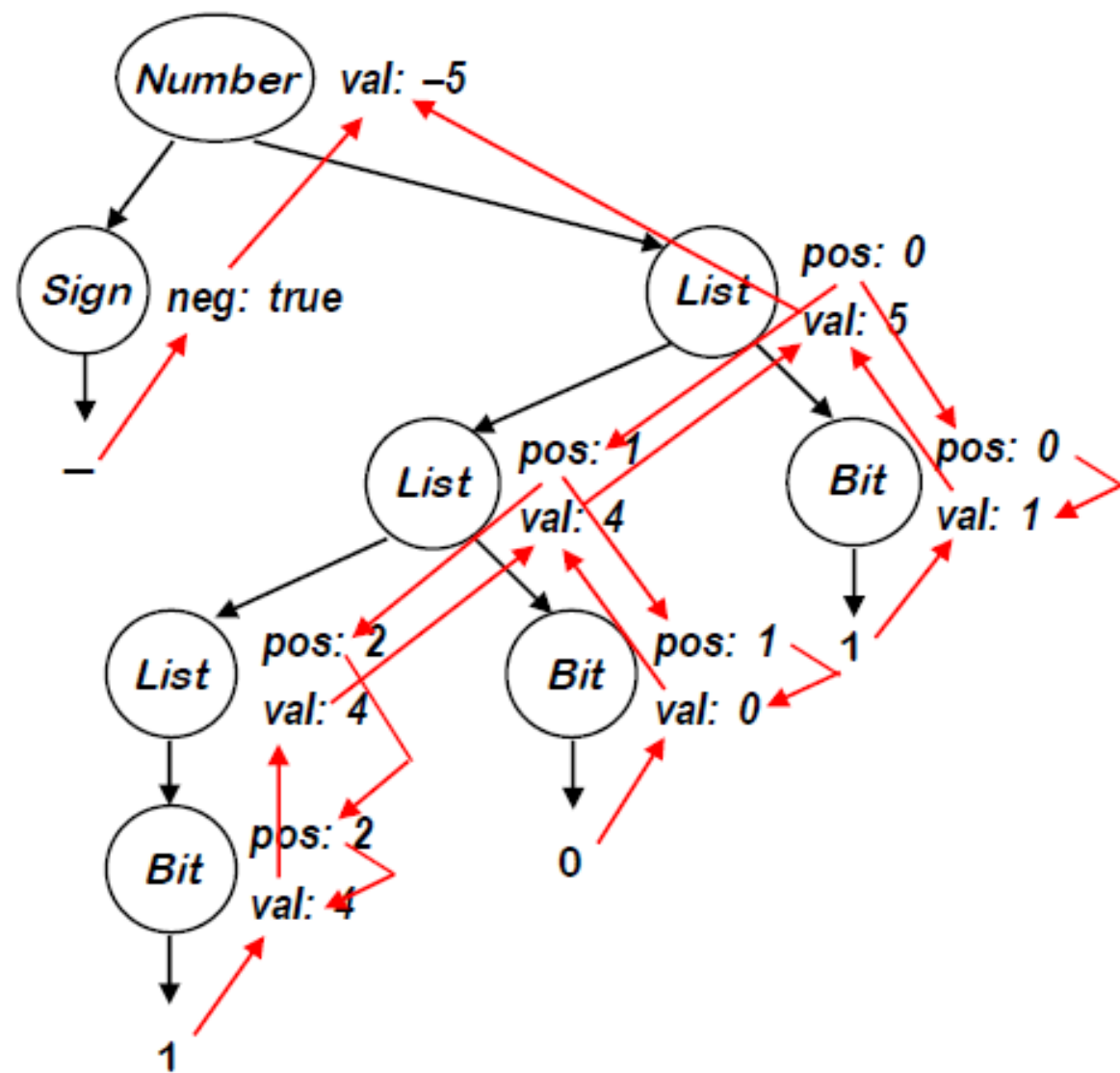
2. Dependency graph



3. Topological sort

1. SIGN.neg
2. LIST₀.pos
3. LIST₁.pos
4. LIST₂.pos
5. BIT₀.pos
6. BIT₁.pos
7. BIT₂.pos
8. BIT₀.val
9. LIST₂.val
10. BIT₁.val
11. LIST₁.val
12. BIT₂.val
13. LIST₀.val
14. NUM.val

Vstup: -101, výstup -5

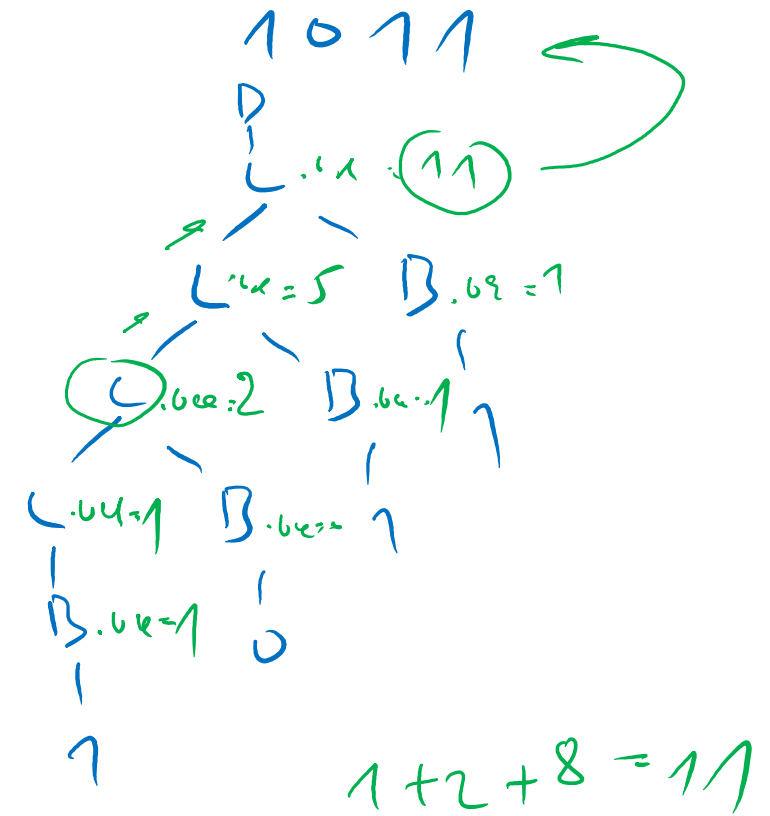


Syntaxí řízení překlad, syntetizované atributy

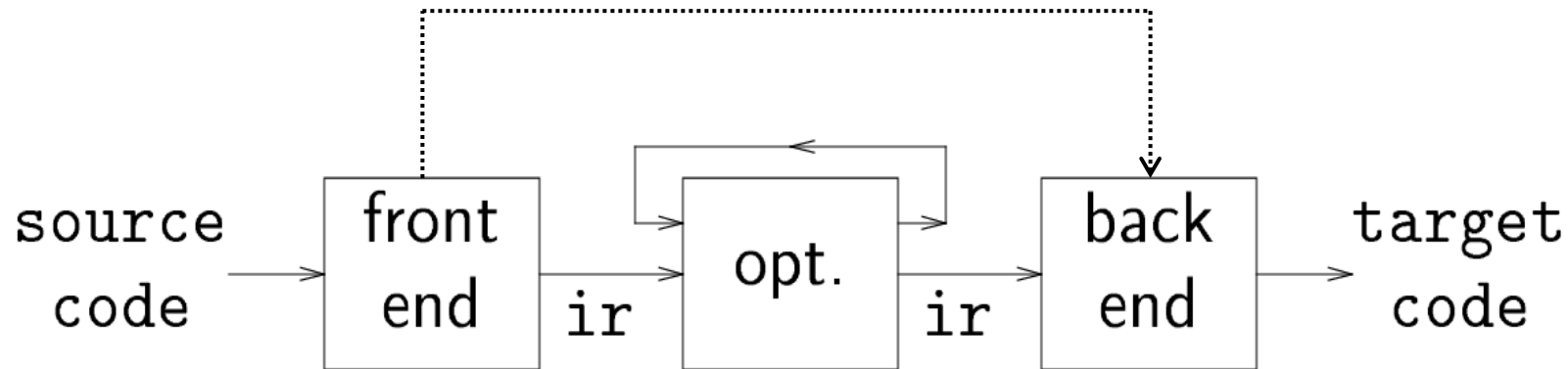
Navrhněte syntaxí řízený překlad pro převod binárního čísla na dekadické. Snažte se přitom použít výhradně syntetizované atributy. Generující gramatika může být například v následujícím tvaru:

$D \rightarrow L \quad \{D.val = L.val\}$
 $\rightarrow L \rightarrow LB \quad \{L.val = L.val * 2 + B.val\} \leftarrow$
 $L \rightarrow B \quad \{L.val = B.val\} \leftarrow$
 $B \rightarrow 1 \quad \{B.val = 1\} \leftarrow$
 $B \rightarrow 0 \quad \{B.val = 0\}$

Funkčnost demonstrujte pro vstup 1011.



Vnitřní forma (mezikód /MK/, intermediální kód)



- Může, ale nemusí existovat
- Může mít jednu nebo více forem (viz následující strany)
- Neplatí zde striktní pravidla, ale volba MK často souvisí s cílovou platformou

Proč se jím zabývat:

- Dvoustupňový vývoj je jednodušší
- Možnost podpory různých cílových platforem
- Uplatnění strojově nezávislých optimalizací

Formální reprezentace mezikódu: souvisí i s typem procesoru

1. Strukturované (grafy, stromy):

- **Implementace:** Abstract Syntax Tree (AST), Direct Acyclic Graph (DAG), Control Flow Graph (CFG), Program Dependence Graph (PGD)
- Akce generují odpovídající grafy stromového typu

2. Nestrukturované, využívající zásobníku

- **Implementace:** Postfix, P-code, Java bytecode, MS CIL (Common IM Lang.)
- Akce organizují zásobník

3. Nestrukturované , využívající *n-tic* (registrové formy)

- **Implementace:** Tříadresový kód, čtveřice apod.
- Akce generují pseudokód

Generování vnitřní reprezentace programu

Miroslav Beneš
Dušan Kolář



Mezikód, příklad č. 1 – aritmetický výraz

Za předpokladu platnosti standardních pravidel pro asociativitu a prioritu operátorů převeďte výraz:

$$a^{*-(b+c)}$$

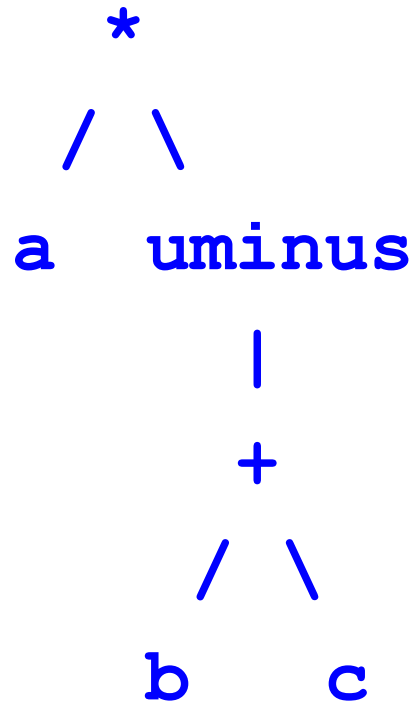
do následujících forem mezikódu:

- a) Abstraktní syntaktický strom
- b) Postfixová notace
- c) Tříadresový kód

Poznámka: MK pro aritmetický výraz reprezentuje nejjednodušší případ, tzv. základní blok

Mezikód, příklad č. 1, řešení

AST:



$a * -(b + c)$

Postfix:

$a \ b \ c \ + \ uminus \ *$ *nebo*
 $b \ c \ + \ uminus \ a \ *$

TAC:

$t1 = b + c$
 $t2 = - \ t1$
 $t3 = a * t2$

Rozdělení procesorů

- **Virtuální:** libovolná architektura (SW)
- **Reálné:**
 - CISC (Complex Instruction Set Computer)
 - RISC (Reduced Instruction Set Computer) – LOAD, STORE
- **Registrové:** mají více registrů, což podporuje bohatější instrukční sadu (překládaný program vede na celkově méně instrukcí), ale vyžaduje jistý čas na dekodování instrukce
- **Zásobníkové:** pracují pouze s vrcholem zásobníku, což vede na jednodušší adresaci, ale obvykle vyžaduje více instrukcí
 - JVM VM: zásobníkový, interpret
 - Dalvik VM (Google): registrový , JIT překladač
 - ART VM: registrový, AOT (od A v.7.0 AOT+JIT)

Registrový a zásobníkový stroj, srovnání

- Registrový (virtuální) procesor:

R1: 5				R1: 5
R2: 10	→	ADD R1, R2, R3	→	R2: 10
R3: 50				R3: 15
R4: 100				R4: 100

- Zásobníkový (virtuální) procesor:

SP: 5		POP 5		SP: 15
10		POP 10		20
20	→	ADD 5, 10, result	→	100
100		PUSH result		

Zásobníkový a registrový procesor, ilustrativní příklad

Instrukce: **B + C - D**

Postfix: B C + D -

Zásobníkový kód:

```
push val B
push val C
add
push val D
sub
```

Instrukce: **A = B**

Postfix: A B =

Zásobníkový kód:

```
push add A
push val B
store
```

B + C - D

```
load r0, B
load r1, C
add r0, r1, r0
load r2, D
sub r0, r2, r0
```

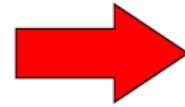
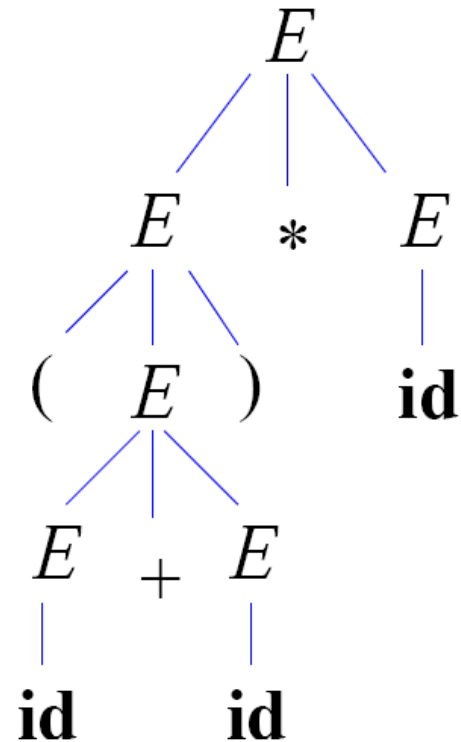
A = B

```
load r0, add A
load r1, val B
store r1, (r0)
```

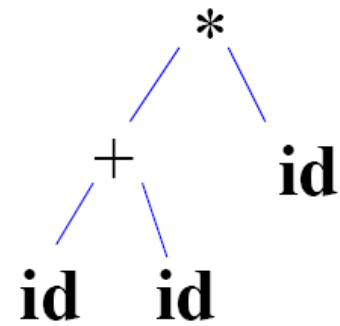
1. Strukturovaná reprezentace

(a + b) * c

Parse Tree



AST



SDT pro AST aritmetických výrazů, S-atributová LR gramatika

$E \rightarrow E1 + T \quad \{ E.\text{node} = \text{Node}(' + ', E1.\text{node}, T.\text{node}); \}$

$E \rightarrow T \quad \{ E.\text{node} = T.\text{node}; \}$

$T \rightarrow (E) \quad \{ T.\text{node} = E.\text{node}; \}$

$T \rightarrow \text{id} \quad \{ T.\text{node} = \text{Leaf}(\text{id}, \text{id}.\text{entry}); \}$

SDT pro AST aritmetických výrazů, L-atributová LL gramatika

$E \rightarrow T A$ { $E.\text{node} = A.s$;
 $A.i = T.\text{node}$; }

$A \rightarrow + T A1$ { $A1.i = \text{Node}('+', A.i, T.\text{node})$;
 $A.s = A1.s$; }

$A \rightarrow \epsilon$ { $A.s = A.i$; }

$T \rightarrow (E)$ { $T.\text{node} = E.\text{node}$; }

$T \rightarrow \text{id}$ { $T.\text{node} = \text{Leaf}(\text{id}, \text{id.entry})$; }

Konstrukce syntaktického stromu, *yacc*

S	→	Expr	$$$ = \$1;$
Expr	→	Expr + Term	$$$ = \text{MakeAddNode}(\$1, \$3);$
		Expr – Term	$$$ = \text{MakeSubNode}(\$1, \$3)$
		Term	$$$ = \$1;$
Term	→	Term * Factor	$$$ = \text{MakeMulNode}(\$1, \$3);$
		Term / Factor	$$$ = \text{MakeDivNode}(\$1, \$3);$
		Factor	$$$ = \$1;$
Factor	→	(Expr)	$$$ = \$1;$
		num	$$$ = \text{MakeNumNode}(\text{token});$
		id	$$$ = \text{MakeIdNode}(\text{token});$

2. Tříadresový kód (TAC)

(vhodný pro registrové stroje)

- Definovaná sada instrukcí, realizujících činnosti na úrovni výrazů a příkazů 
 - Aritmetické a logické operace, přiřazení, indexace, adresace, řízení běhu (skoky, návěští), volání funkcí a návraty
- Instrukce má vždy pouze jeden operátor s jednoznačnou aritou (počtem operandů) a nemá nikdy více jak 3 operandy:

$x := y \text{ op } z$

kde x, y, z mohou být jména, konstanty nebo dočasné proměnné překladače. Výraz typu:

$d := x + y * z$

je pak přeložen jako :

$t1 := y * z$

$t2 := x + t1$

$d := t2$

Příklady instrukcí TAC

Operace a práce s pamětí

$x := y \text{ op } z$ přiřazení (uložení)

$x := \text{op } y$ unární přiřazení

$x := y$ kopírování

Skoky

`goto L` nepodmíněný skok

`if x relop goto L` podmíněný skok

Procedury/funkce

`param x`

`call p n`

`return y`

Pole

$x := y[i]$

$x[i] := y$

Ukazatele

$x := \&y$

$x := *y$

$*x = y$

Implementace příkazů TAC: čtveřice

$a := b * -c + b * -c$

#	Op	Arg1	Arg2	Res
(0)	uminus	c		t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	:=	t5		a

Implementace příkazů TAC: trojice

a := b * -c + b * -c

#	Op	Arg1	Arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	:=	a	(4)

Implementace příkazů TAC: nepřímé trojice

$a := b * -c + b * -c$

#	Stmt	
(0)	(14)	-->
(1)	(15)	-->
(2)	(16)	-->
(3)	(17)	-->
(4)	(18)	-->
(5)	(19)	-->

#	Op	Arg1	Arg2
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	:=	a	(18)

Program

Zásobník trojic

Generování mezikódu, syntaxí řízené definice (SDD)

Atributy pro výrazy (*expressions*) E :

E.place : **adresa** paměťového místa s hodnotou E

E.code : instrukce, vyhodnocující E

Viz následující slajd



Atributy pro příkazy (*statements*) S :

S.begin : **adresa** první instrukce kódu, odpovídajícímu S

S.after : **adresa** první instrukce kódu, následujícímu po S

S.code : instrukční soubor, odpovídající S

Existence více adres činí zpracování příkazů problematické

TAC, SDD

S -> id:=E	S.code := E.code gen(id.place ':=' E.place)	Výstup: ASS(E.place, - , id.place)
E -> E1+E2	E.place := newtemp; E.code := E1.code E2.code gen(E.place ':=' E1.place '+' E2.place)	Výstup: ADD(E1.place, E2.place, E.place)
E -> E1*E2	E.place := newtemp; E.code := E1.code E2.code gen(E.place ':=' E1.place '*' E2.place)	Výstup: MUL(E1.place, E2.place, E.place)
E -> -E1	E.place := newtemp; E.code := E1.code gen(E.place ':=' 'uminus' E1.place)	Výstup: UMINUS(E1.place, - ,E.place)
E -> (E1)	E.place := E1.place; E.code := E1.code;	
E -> id	E.place := id.place; E.code := " "	

Problematika příkazů a logických výrazů

Zdrojový kód

if $x + 2 > 3 * (y - 1) + 4$ **then** $z := 0$;

Odpovídající mezikód:

$t1 := x + 2$

$t2 := y - 1$

$t3 := 3 * t2$

$t4 := t3 + 4$

if $t1 \leq t4$ goto L

$z := 0$

L: ...

Přímočarý způsob práce s logickými výrazy

Syntaktické pravidlo: **BExp -->E1 relop E2**

Odpovídající mezikód:

```
            t1 <-- hodnota E1
            t2 <-- hodnota E2
        t3 := TRUE
        if t1 relop t2 goto L
        t3 := FALSE
    label L: ...
```

Nedostatky:

- Velké množství pomocných proměnných
- Častý přístup do paměti
- Nemožnost zkráceného vyhodnocování výrazů

Podmíněné příkazy

Syntaktické pravidlo:

S --> if E then S1 else S2

Sémantická pravidla:

```
{      S.begin := newlabel();
      S.after  := newlabel();
      S.code   := gen('label' S.begin)      |
      E.code   |
      gen('if' E.place '=' ' 0' ' goto' S2.begin) |
      S1.code  |
      gen('goto' S.after) |
      S2.code  |
      gen('label' S.after)
}
```

Cykly

Syntaktické pravidlo: **S --> while E do S1**

Struktura generovaného kódu:

```
L1 :    Vyhodnoť E
      if (E == FALSE) goto L2
      Kód pro S1
      goto L1
L2 :    ...
```

Sémantická pravidla:

```
{    S.begin := newlabel();
    S.after  := newlabel();
    S.code  := gen('label' S.begin)
        E.code
        gen('if' E.place '=' ' 0' 'goto' S2.after)
        S1.code
        gen('goto' S.begin)
        gen('label' S.after)
}
```


Generování adres pomocí markerů

E --> E1 or M E2

- Marker nemá žádnou syntaktickou akci, tj. $M \rightarrow \varepsilon$

Zdrojový kód: **E = a<b or c<d and e<f**

Odpovídající mezikód:

```
100 : if a < b goto __E_true__  
101 : goto 102  
102 : if c < d goto 104  
103 : goto __E_false__  
104 : if e < f goto __E_true__  
105 : goto __E_false__
```

E.truelist = {100; 104}

E.falselist = {103; 105}

E.true : instrukce

E.false: instrukce

Přímočará (intuitivní, naivní) adresace

Vede na nepodmíněné skoky a ne_monotonní adresaci:

Zdrojový kód: **while a < b do**
 if x < y then S endif
 endwhile

Odpovídající mezikód:

L1 : if a < b goto L2
 if x < y goto L3
 goto L4
L3 : S.code
L4 : goto L1
L2 :

Tento nedostatek lze opět řešit pomocí markerů na úrovni gramatického pravidla (*backpatching*)

S --> if E then M1 S1 N else M2 S2
S --> while M1 E do M2 S1

3. Postfixová reprezentace

(vhodná pro zásobníkové stroje)

- **Infix**

- Způsob, jakým běžně zapisujeme aritmetické výrazy:

$$a+b*(c^d-e)^{(f+g*h)}-i$$

- **Postfix**

- Způsob, vhodný pro interpretaci aritmetického výrazu zásobníkovým strojem:

$$abcd^e-fgh*+^*+i-$$

Naplnění zásobníku formou postfixové reprezentace

Gramatika:

$E \rightarrow E + T \{ \text{push } + \}$

$E \rightarrow E - T \{ \text{push } - \}$

$E \rightarrow T$

$T \rightarrow T * F \{ \text{push } * \}$

$T \rightarrow T / F \{ \text{push } / \}$

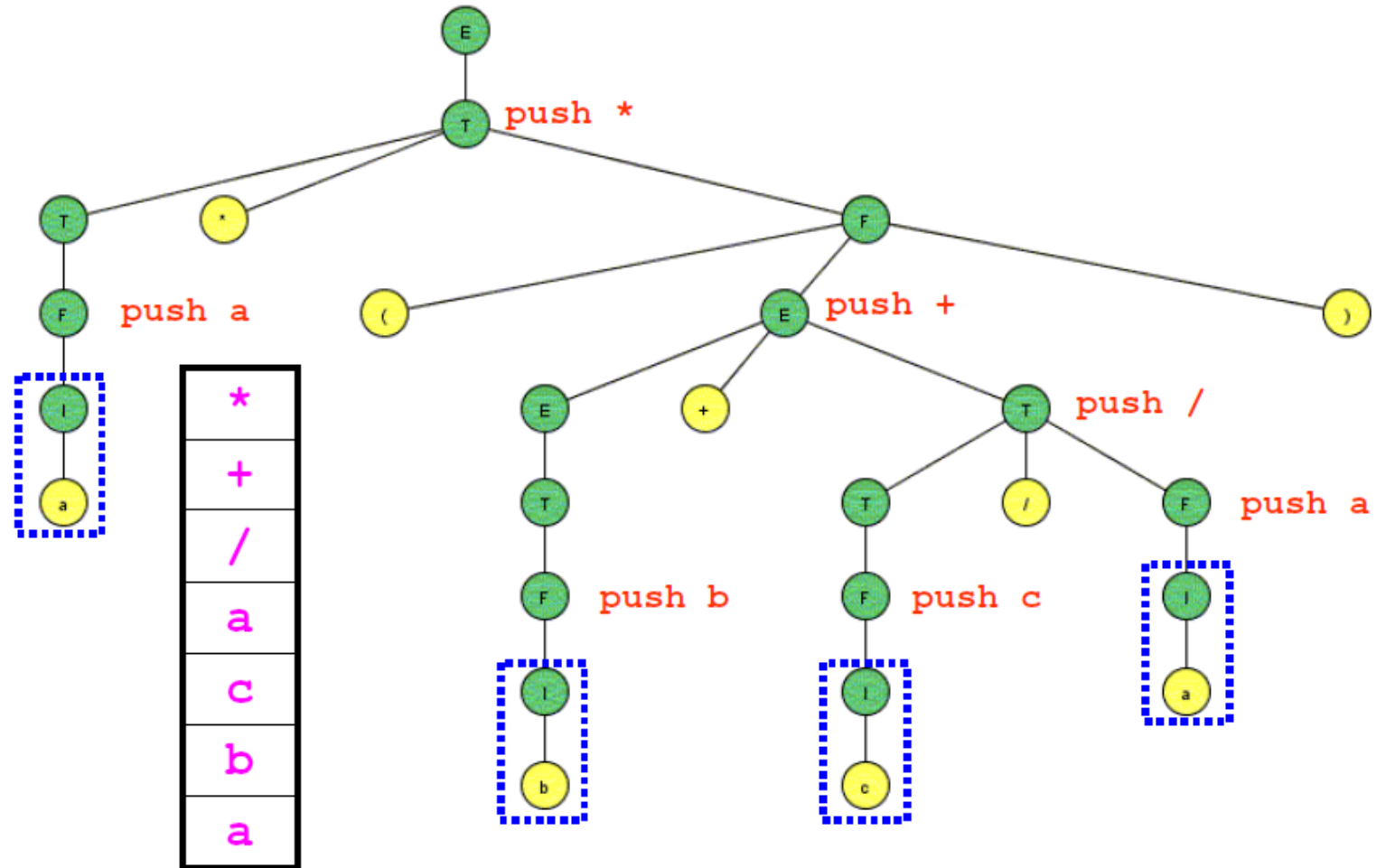
$T \rightarrow F$

$F \rightarrow i \{ \text{push } i \}$

$F \rightarrow (E)$

Vstup:

$a*(b+c/a)$



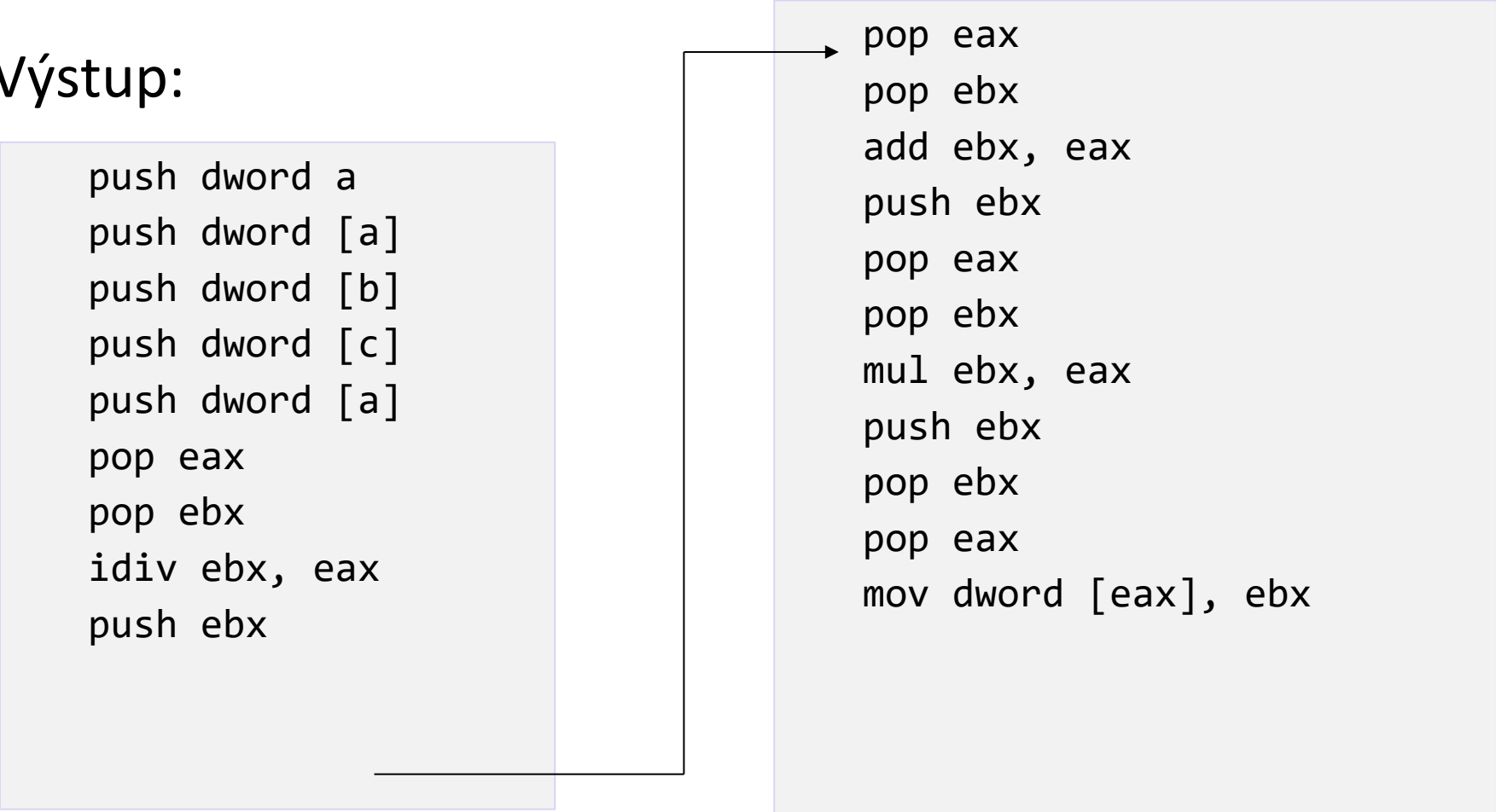
Generování cílového kódu z postfixu

Vstup: $a = a * (b + c / a)$

Výstup:

```
push dword a
push dword [a]
push dword [b]
push dword [c]
push dword [a]
pop eax
pop ebx
idiv ebx, eax
push ebx
```

```
pop eax
pop ebx
add ebx, eax
push ebx
pop eax
pop ebx
mul ebx, eax
push ebx
pop ebx
pop eax
mov dword [eax], ebx
```



Mezikód, příklad č. 2 – logický výraz

Za předpokladu platnosti standardních pravidel pro asociativitu a prioritu operátorů převeďte výraz:

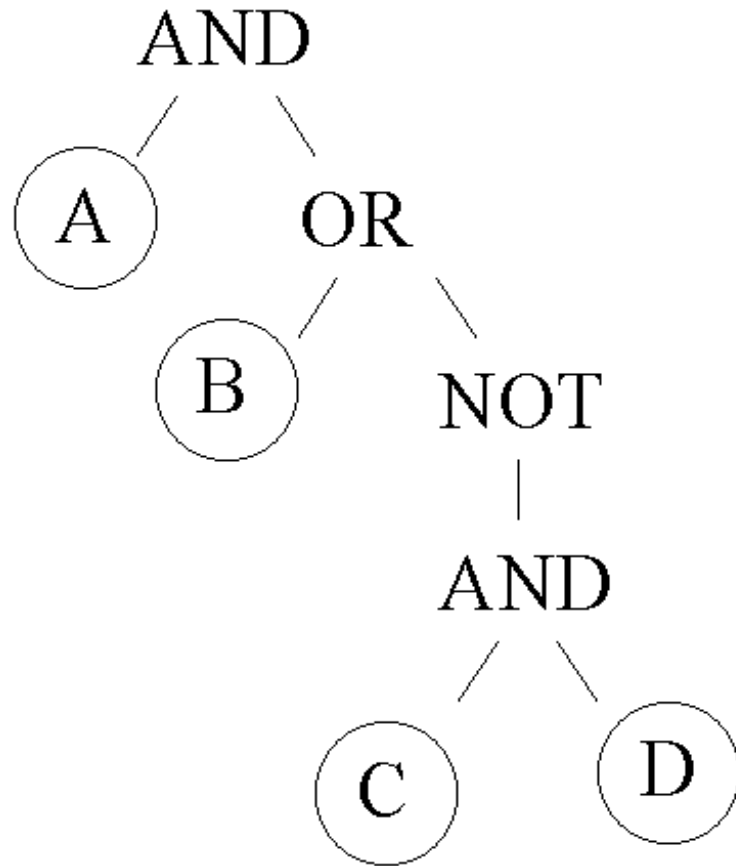
A and (B or not (C and D))

do následujících forem mezikódu:

- Abstraktní syntaktický strom (AST nebo DAG)
- Tříadresový kód v libovolné formě reprezentace

Mezikód, příklad č. 2, řešení

1) AST:



A and (B or not (C and D))

2) TAC

T1 := C and D

T2 := not T1

T3 := B or T2

T4 := A and T3

TAC, zkrácená forma

if not A goto FALSE

if B goto TRUE

if not C goto TRUE

if D goto FALSE

TRUE:

...

FALSE:

...

Mezikód, příklad č. 3, řídicí příkaz

Převeďte následující příkaz do TAC:

```
if (a < b + c)
```

```
    a = a - c;
```

```
c = b * c;
```


Mezikód, příklad č. 3, řešení

```
if (a < b + c)
    a = a - c;
c = b * c;
```



```
t1 = b + c;
t2 = a < t1;
FJP L0;
t3 = a - c;
a = t3;
L0: t4 = b * c;
c = t4;
```